

LECTURE

Introduction to HCP Terraform





HCP Terraform

What is HCP Terraform?

HCP Terraform is a hosted service that helps teams use Terraform to provision infrastructure. HCP Terraform can help you collaborate, manage remote state, and automate workflows.

The Value of HCP Terraform



Secure Remote State Storage

Single source of truth
Encrypted at rest
State locking built-in



Team Collaboration

Shared workspaces
Role-based access control
Audit logs



Remote Execution

Consistent environment
No local dependencies
Reliable runs



Governance & Policy

Policy enforcement
Cost estimation
Drift detection

Comparing Terraform with HCP Terraform



Terraform Community

- ✓ Define your infrastructure as code
- ✓ Free and "code-available"
- ✓ Rich set of integrations (providers)
- ✓ Reusable modules (public registry)
- ✓ Ability to plan out your changes
- ✓ Local execution and control

☁ HCP Terraform adds...

- + Managed remote state storage
- + Consistent execution environments
- + Built-in team collaboration
- + Role-based access control (RBAC)
- + Policy enforcement (Sentinel/OPA)
- + Cost estimation before apply
- + Audit logs and compliance
- + Private module registry

How Does HCP Terraform Impact Me?



💛 Your Code Still Works!!

- Everything you've learned about Terraform - the `resource` blocks, the `data` sources, the modules you've created - it all works exactly the same with HCP Terraform
- HCP Terraform doesn't change the Terraform language - it changes where and how your code runs

💻 Leverage existing Workflows:

- Your CLI commands (`terraform plan`, `terraform apply`) work the same way
- Version control, CI/CD pipelines, and automation all integrate seamlessly
- You gain collaboration features, audit logs, and managed state without changing your daily workflow

🚀 Adopt at Your Own Pace:

- Start with just remote state storage and add features over time
- Keep using local execution if you want - or switch to remote when you're ready
- Whether you're a solo practitioner or part of a large team, HCP Terraform scales with you

The HCP Terraform Workflow



Plan runs remotely in HCP Terraform
See changes before applying



Write Terraform configuration locally
Same .tf files you already know

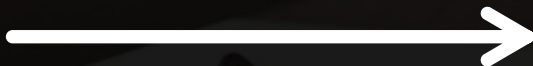
Apply runs remotely in HCP Terraform
Infrastructure created/updated

Where Does Code Execution Happen?

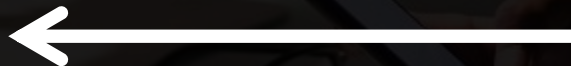


Your Local Machine

Commands Sent



Results Returned to CLI



Execution Happens
Here

Execution Modes: Where Runs Happen



Remote Execution

VS

Local Execution

- ✧ Plans and applies run in HCP Terraform
- ✧ Recommended for teams
- ✧ Most HCP features available
- ✧ Results streamed back to local CLI

- ✧ Plans and applies run on your machine
- ✧ State still stored remotely in HCP Terraform
- ✧ Limited access to HCP features

Connecting to HCP Terraform



```
1 terraform {
2   cloud {
3     organization = "my-org"
4     hostname = "app.terraform.io"
5
6     workspaces {
7       project = "networking-development"
8
9       tags = {
10        layer = "networking"
11        source = "cli"
12      }
13    }
14  }
15 }
```

cloud block tells Terraform to use HCP

Point to your organization (auth required)

Configure what workspace/project to use

HCP Terraform Tiers



Free Tier

State storage and basic features

Includes 500 managed resources at no charge



Essentials

Access to More Features

Includes Stacks, Unlimited Applies, etc.



Standard

For Enterprises

Drift Detection, Golden Patterns, Audit Events, Policy Enforcement



Premium

Multi-Cloud Enterprise

Private VCS, Run Tasks, Module Revocation

***Subject to Change – Check for Updates**

How Do I Sign Up?



Visit the HCP Terraform sign-up page and follow the prompts to create a free HCP Terraform account.



<https://app.terraform.io>

The sign-in form for HCP Terraform is displayed on a white background. At the top is the HCP Terraform logo. Below it is the title "Sign in to HCP Terraform" followed by the instruction "Enter your email or username to continue." There is a text input field labeled "Email or Username" with a blue border. Below the input field is a blue button labeled "Continue". A horizontal line with the word "OR" in the center separates this from another option. Below the line is a button labeled "Sign in with SSO". At the bottom of the form, there is a link that says "Don't have an account? Create an account →".

Key Takeaways



01



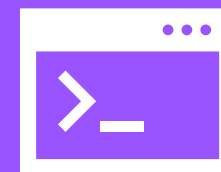
HCP Terraform is a managed platform that extends Terraform

02



Same Terraform code - different execution environment

03



Remote execution is default; local execution available for flexibility

LECTURE

Getting Connected and Authenticated



The Gap We Need to Bridge



```
$ terraform plan  
$ terraform apply
```

How Do These Talk
to Each Other?

A purple cloud shape containing the Google Cloud logo and the text 'HCP Terraform'.

HCP Terraform

You need to authenticate before HCP Terraform trusts your CLI

Connecting to HCP Terraform



Step 1

Add your `cloud` block to your Terraform configuration

```
1 terraform {  
2   cloud {  
3     organization = "my-org"  
4     hostname = "app.terraform.io"  
5  
6     workspaces {  
7       project = "networking-development"  
8  
9       tags = {  
10        layer = "networking"  
11        source = "cli"  
12      }  
13    }  
14  }  
15 }
```

`organization` argument = your org name

`workspaces` block with the two approaches:

- `name/project` = target a specific workspace/project
 - `tags` = targets workspaces matching tag(s)

Authenticating with terraform login



Step 2

TERMINAL

```
$ terraform login
Terraform will request an API token for app.terraform.io using your
browser.

If login is successful, Terraform will store the token in plain text in
the following file for use by subsequent commands:
  /Users/bk/.terraform.d/credentials.tfrc.json

Do you want to proceed?
  Only 'yes' will be accepted to confirm.

Enter a value: yes
```


Authenticating with terraform login



Tokens

Create an API token

Your API tokens can be used to access the HCP Terraform API and perform all the actions your user account is entitled to. For more information, see the [user API tokens documentation](#).

Treat these tokens like passwords, as they can be used to access your account without a username, password, or two-factor authentication.

Tokens (1)



terraform login Expires today at 10:40 AM



i Copy the token, then return to your terminal and paste it into your Terraform login prompt. HCP Terraform will not display the token again, so store it securely.

FCApkzYtVEW0xg.atlasv1.4clxpkbzVvHqgebEhAPeiVhCMYuU1kUF090Nao4t0Ubo4uMv3gDEsgy9JYHBExsLwEw



Created just now by user bkrausen

Token has not been used



Authenticating with terraform login



credentials.tfrc.json — Edited

```
{
  "credentials": {
    "app.terraform.io": {
      "token": "FCApkzYtVEW0xg.atlasv1.4clxpkbzVvHqgebEhAPeiVhCMYuU1kUF090Nao4t0Ubo4uMv3gDEsgy9JYHBExsLwEw"
    }
  }
}
```

TERMINAL

Generate a token for app.terraform.io, then paste it into the prompt.

Terraform will store the token in plain text in the following file for use by subsequent commands:

/Users/bk/.terraform.d/credentials.tfrc.json

Token for app.terraform.io:
Enter a value:

Paste Token Here

Authenticating with terraform login



```

Welcome to HCP Terraform!

Documentation: terraform.io/docs/cloud

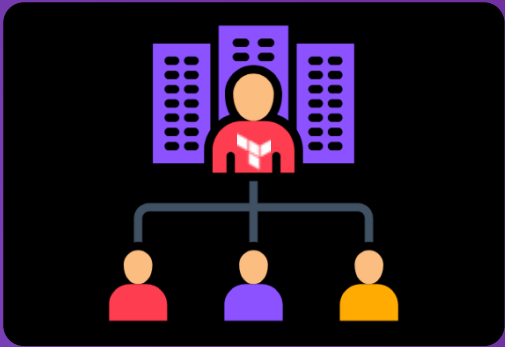
New to HCP Terraform? Follow these steps to instantly apply an example configuration:

$ git clone https://github.com/hashicorp/tfc-getting-started.git
$ cd tfc-getting-started
$ scripts/setup.sh

```



HCP Terraform - API Token Types



Organization Tokens

- Used to Manage Teams, Membership, and Workspaces
- Does not have permission to run plans and applies



Team Tokens

- Often used by automated services, like CI/CD pipelines
- Can run Terraform operations (plan/apply)



User Tokens

- Same permission level as your user account
- Only type of token that can access multiple organizations



Audit Tokens

- Provides read-only access
- Used for accessing the audit trail API

HCP Terraform - Environment Variable Alternative



As an alternative to using `terraform login`, you can set the token using the env:

```
TF_TOKEN_app_terraform_io=<token>
```

Why is this important to know?

CI/CD pipelines and automation will likely need this where interactive browser login isn't possible

LECTURE

Understanding HCP Terraform Workspaces



These Are NOT the Workspaces You Know



CLI Workspaces



Same Configuration,
Multiple State Files



Managed locally with
terraform
workspace

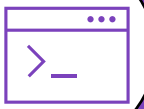


Lightweight state
isolation



HCP Terraform Workspaces

Complete working
environments



Config + state +
variables + settings +
credentials + run
history



Managed through HCP
Terraform (UI, CLI, or
API)



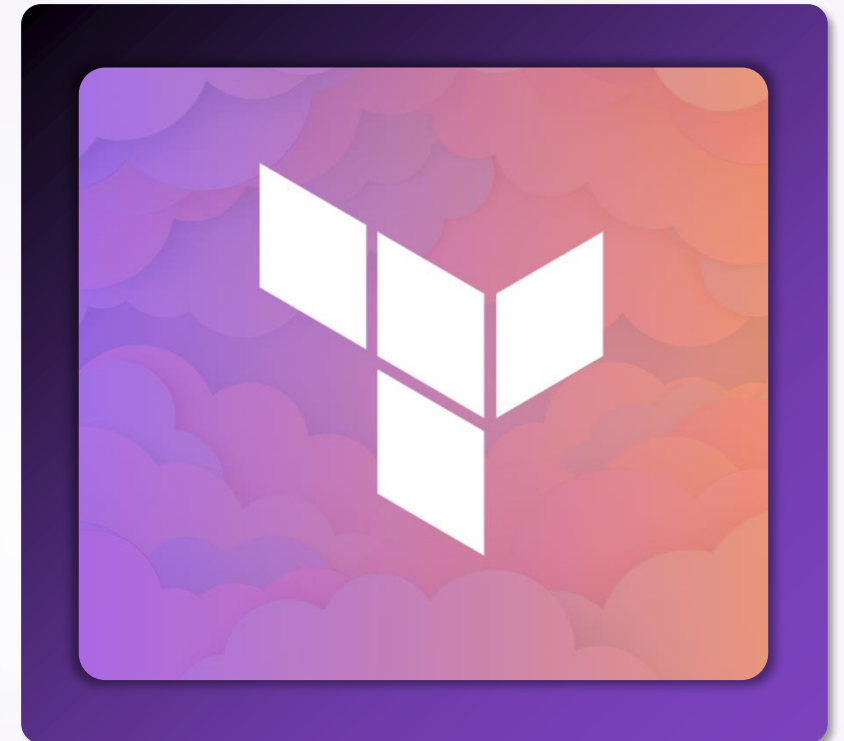
Same Name, Fundamentally Different Concept



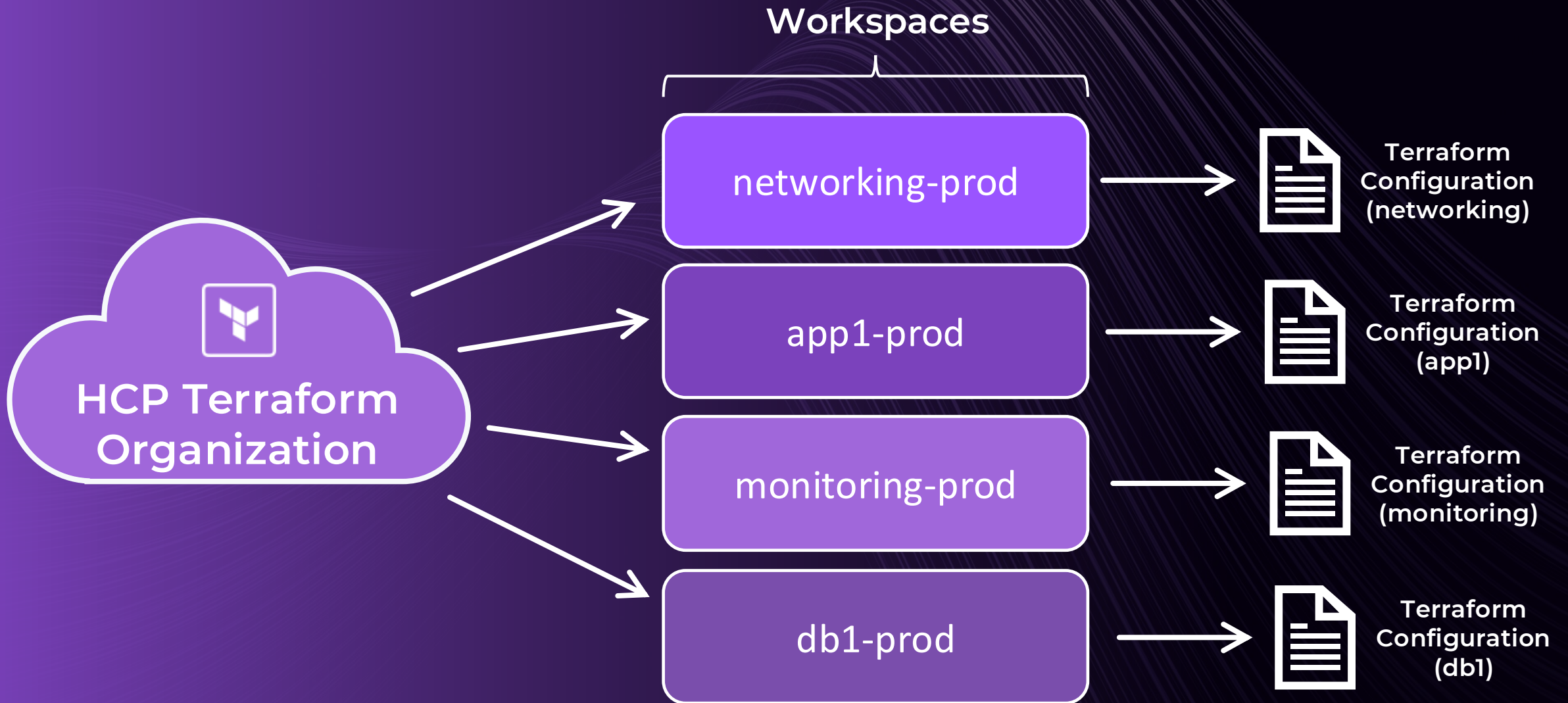
Introduction to HCP Terraform Workspaces

A workspace is everything Terraform needs to manage a collection of infrastructure.

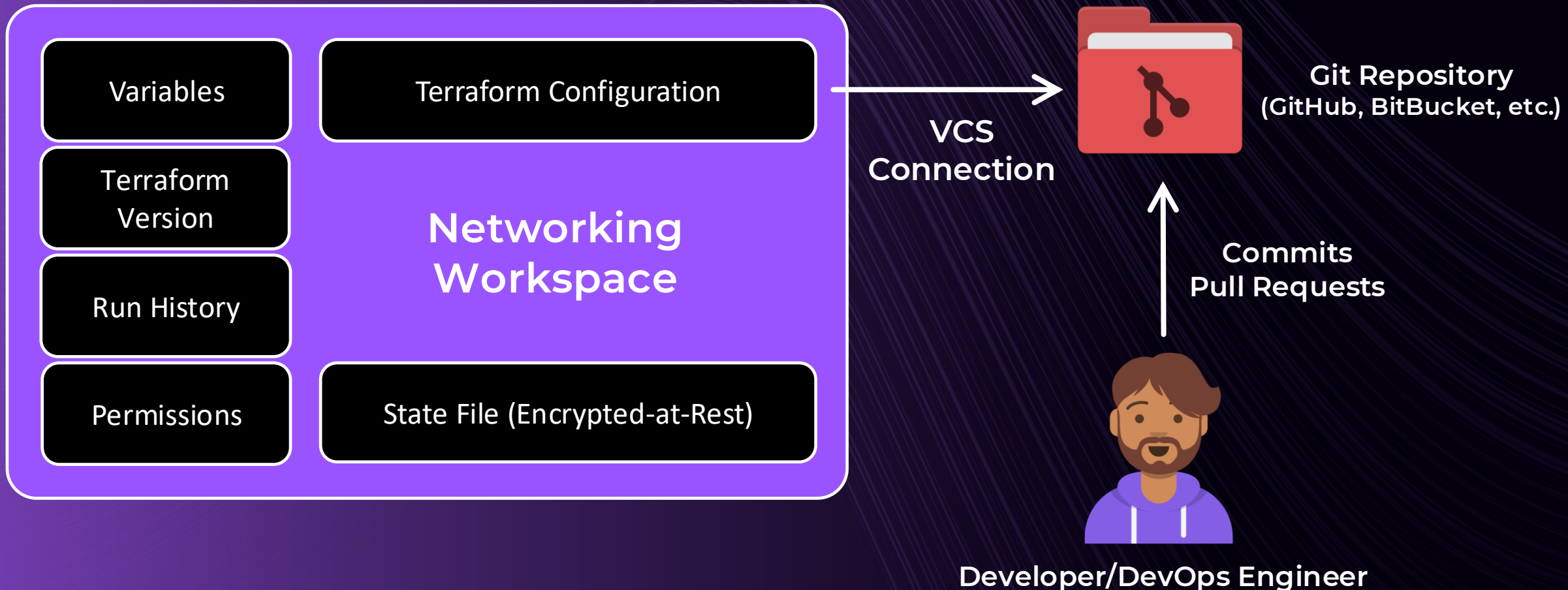
- ✧ Terraform configuration (linked via VCS or uploaded via CLI)
- ✧ State file (with version history and rollback capability)
- ✧ Variables (Terraform variables + environment variables, with sensitive marking)
- ✧ Run history (full audit trail of every plan/apply)
- ✧ Execution settings (mode, Terraform version, auto-apply)
- ✧ Team access and permissions



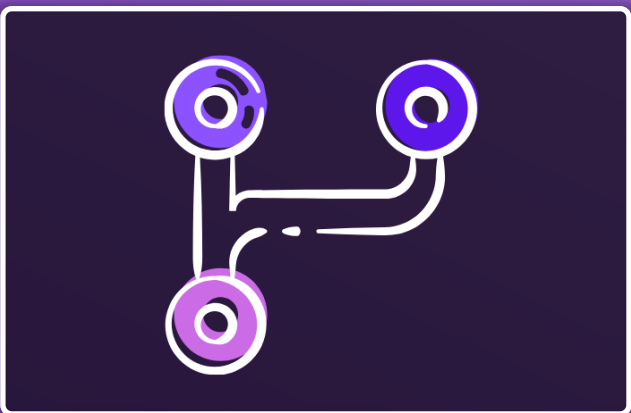
HCP Terraform Workspaces



HCP Terraform Workspaces



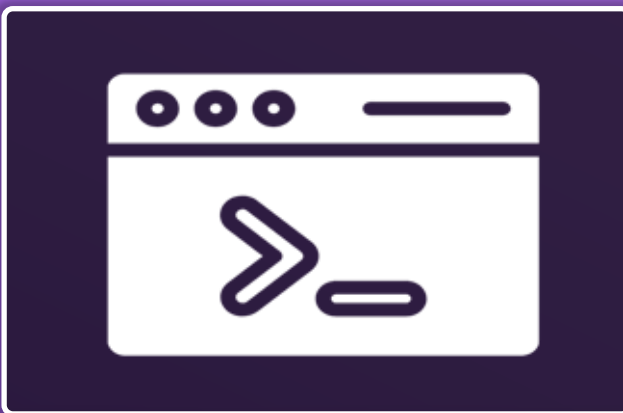
Workspace Workflow Types



Version Control Workflow

Trigger runs based on changes to configuration in repositories

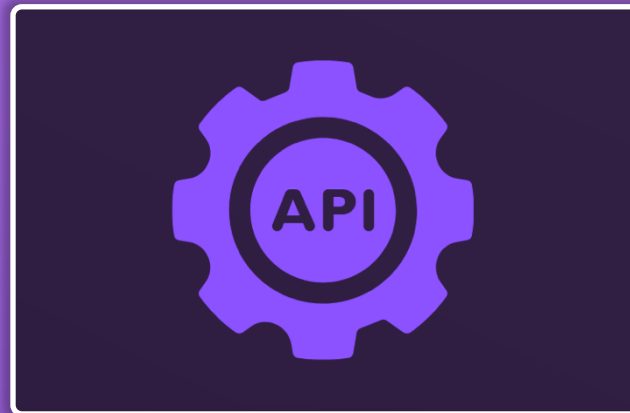
Best for those who need traceability and transparency



CLI-Driven Workflow

Trigger runs in a workspace using the Terraform CLI

Best for those comfortable with Terraform CLI



API-Driven Workflow

Trigger runs using the HCP Terraform API

Best for those with custom integrations and pipelines

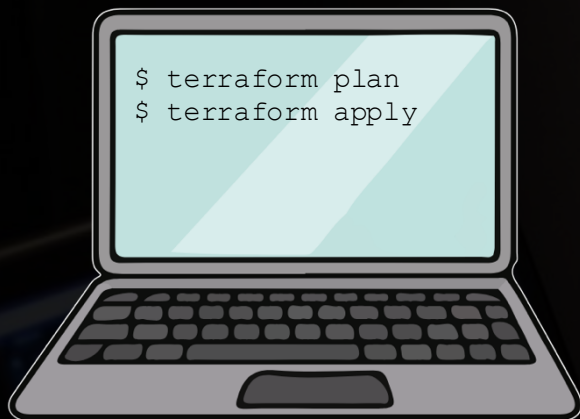
Remote Operations



Terraform runs managed by HCP
Terraform are called **remote operations**

- ✧ Your plans and applies occur on HCP Terraform's infrastructure.
- ✧ You and your team have the ability to review and collaborate on runs within the app.
- ✧ Full feature access (Sentinel, cost estimation, notifications)

Local Execution Mode



You can disable remote operations for any workspace by changing its Execution Mode to **Local**

- ✧ Plan and apply run on your machine
- ✧ HCP Terraform is **only** used to store and synchronize state.
- ✧ Workspace variables and variable sets are NOT evaluated

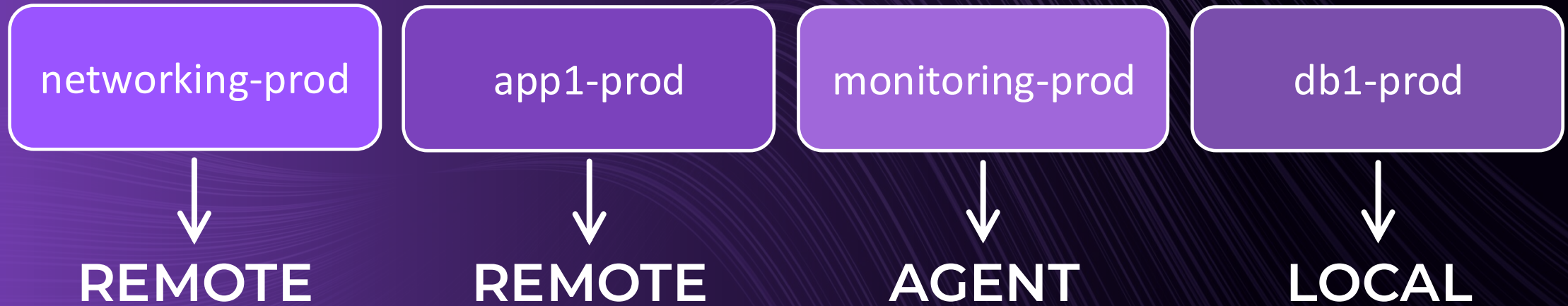
Agent Mode



Run your own lightweight HCP Terraform agents to run Terraform in isolated, private, or on-premises infrastructure.

- ✧ Plan and apply run on a self-hosted agent
- ✧ For private/on-premises infrastructure HCP Terraform can't reach directly
- ✧ HCP Terraform and Terraform Enterprise use the same agent instance to evaluate policies and run Terraform operations

Execution Mode



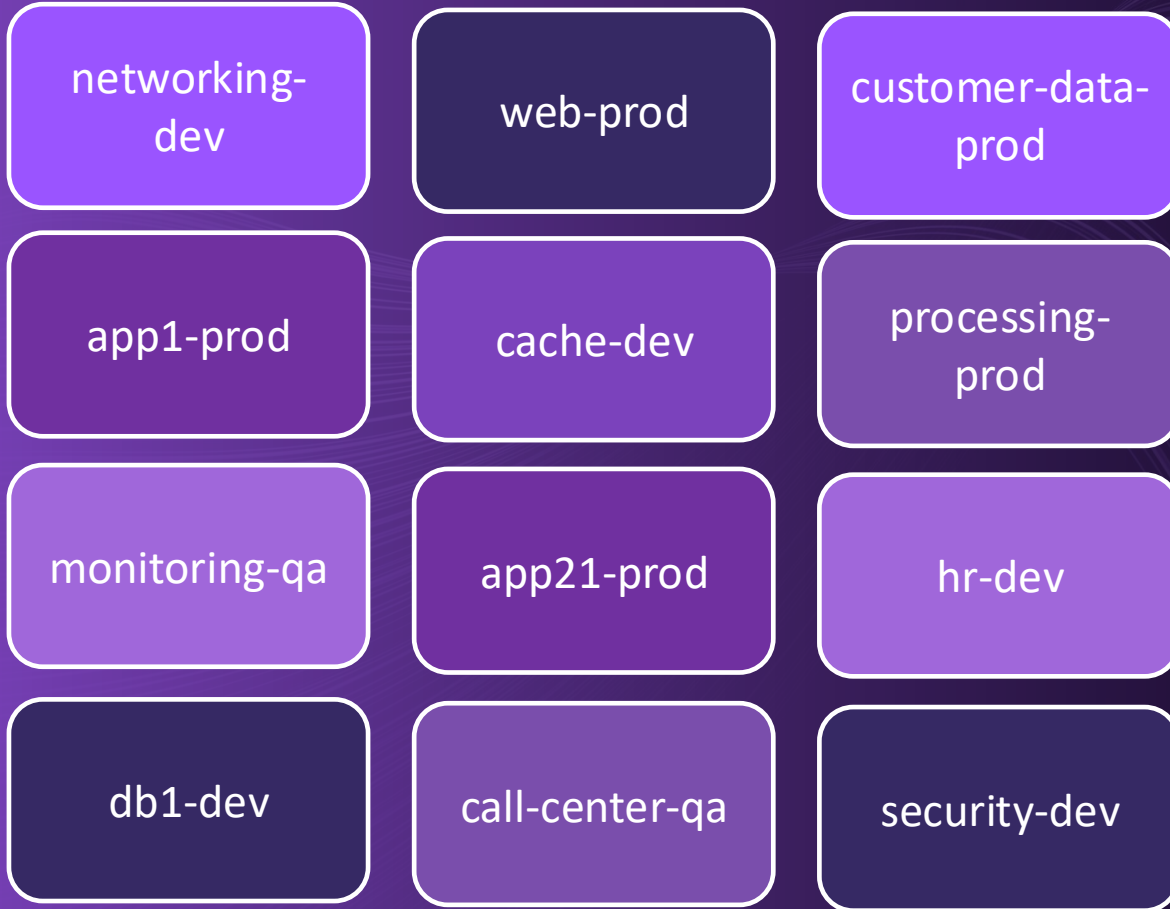
Execution mode is configured per workspace

LECTURE

Organizing Workspaces with Projects



Workspace Sprawl



As Your Organizations Grow:

- ✦ Workspaces Become Unmanageable
- ✦ Making Changes One Workspace at a Time
- ✦ No Clean Way to Separate Team Boundaries
- ✦ Duplicating Efforts

Workspace Organization



Development Project

Quality Assurance Project

Production Project

cache-dev

networking-
dev

hr-dev

db1-dev

security-dev

monitoring-qa

call-center-qa

processing-
prod

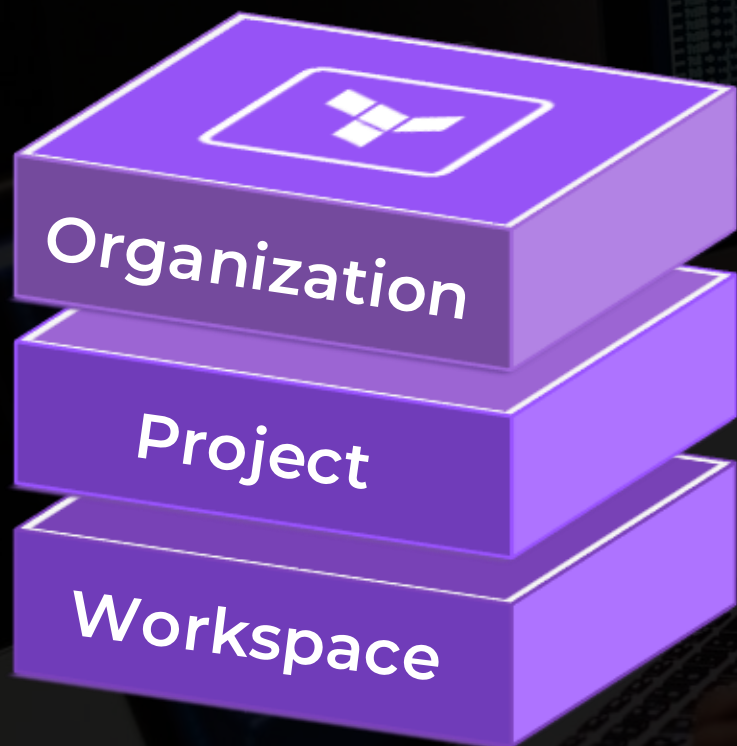
customer-data-
prod

web-prod

app21-prod

app1-prod

Projects – The Organizational Layer



✧ ORGANIZATION

Your top-level account in HCP Terraform. It holds all your teams, projects, and workspaces, and is where you set organization-wide settings.

✧ PROJECT

Organizes related workspaces together. Lets you scope team permissions, variable sets, and execution modes to a collection of workspaces. Every workspace must belong to exactly one project.

✧ WORKSPACE

The individual unit that manages a specific set of infrastructure. Each workspace contains its own Terraform configuration, state file, variables, and run history.

Projects



- ✦ Every workspace belongs to exactly one project - can't belong to two, can't belong to none
- ✦ Default Project - every org gets one automatically; you can rename it but can't delete it
- ✦ Scoped team permissions - grant a team access to an entire project instead of workspace-by-workspace (least privilege)
- ✦ Project-level variable sets - apply variables (like cloud credentials) to all current and future workspaces in the project automatically

Projects



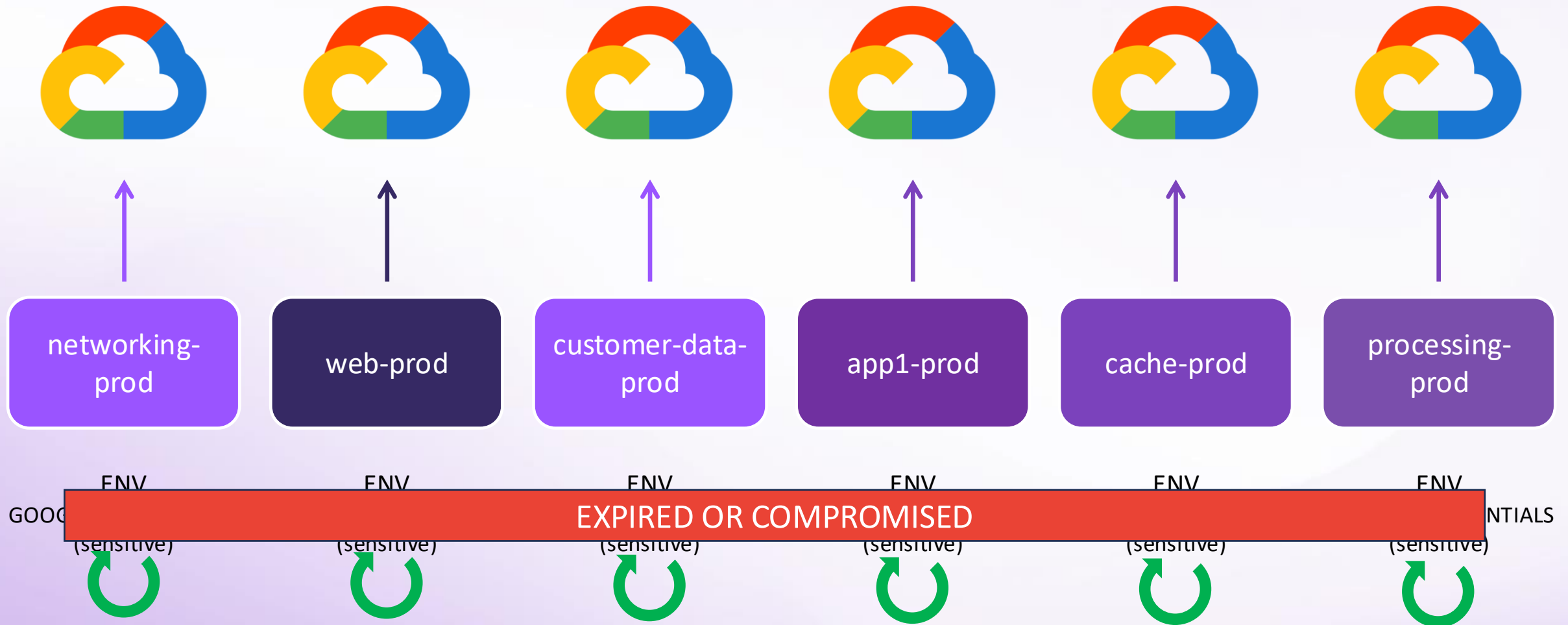
- ✦ Execution mode inheritance - set it at the project level; workspaces inherit from their project, which inherits from the org
- ✦ You can move workspaces between projects and specify a project in the cloud block (Terraform 1.6+)
- ✦ Only empty projects can be deleted

LECTURE

Sharing Variables with Variable Sets



Variable Repetition



Organize Your Variables



Variable Set

Production Cloud

AWS_SECRET_KEY_ID

AWS_SECRET_ACCESS_KEY

AWS_REGION



networking
-prod

web-prod

app1-prod

cache-prod

Variable Set

Database Settings

DB_WRITE_CAPACITY=1

DB_READ_CAPACITY=1



qa-database-project



test-db-project



Variable Set

VM-Size

QA_SIZE=m5.large

TEST_SIZE=m6.2xlarge

PROD_SIZE=M6.4xlarge



networking
-prod

web-prod

app1-prod

cache-prod

customer-
data-prod

processing-
prod

Variable Set

Audit Configs

PATH=/var/log/app

ROTATE=7d



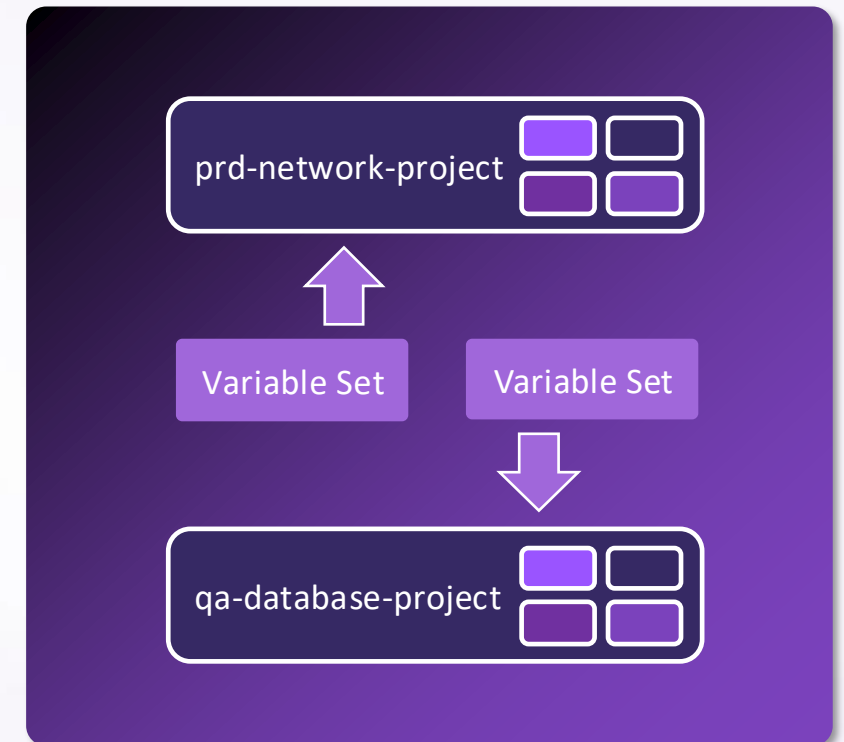
All Projects and
Workspaces



What Are Variable Sets?

Variable sets let you reuse variables in an efficient and centralized way, so you can set variable values once and use them in multiple workspaces.

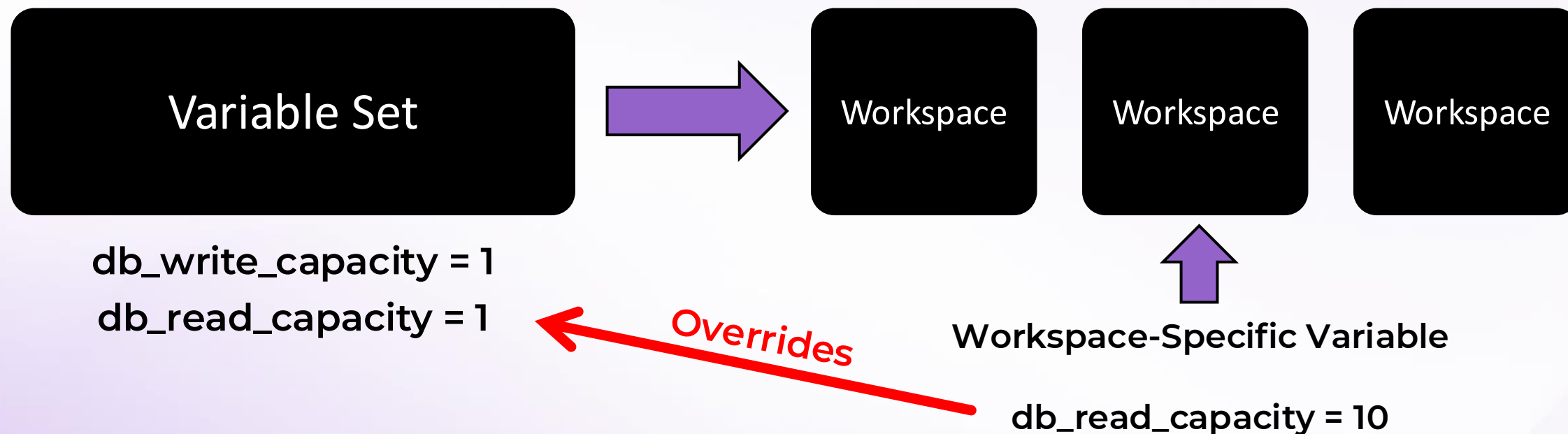
- Reusable collections of variables you define once and apply to many workspaces
- Can contain both Terraform variables and environment variables
- Update a value once and changes propagate to all associated workspaces automatically
- Workspace-specific variables can override variable set values



Variable Set Scopes



Variable Precedence



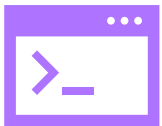
HCP Terraform will always use workspace-specific variables over any variables defined in variable sets applied to the workspace.



Variable Precedence



Priority variable sets can be flagged to override more specific scopes



CLI flags and `TF_VAR_` environment variables override workspace and variable set values in CLI-driven workflows



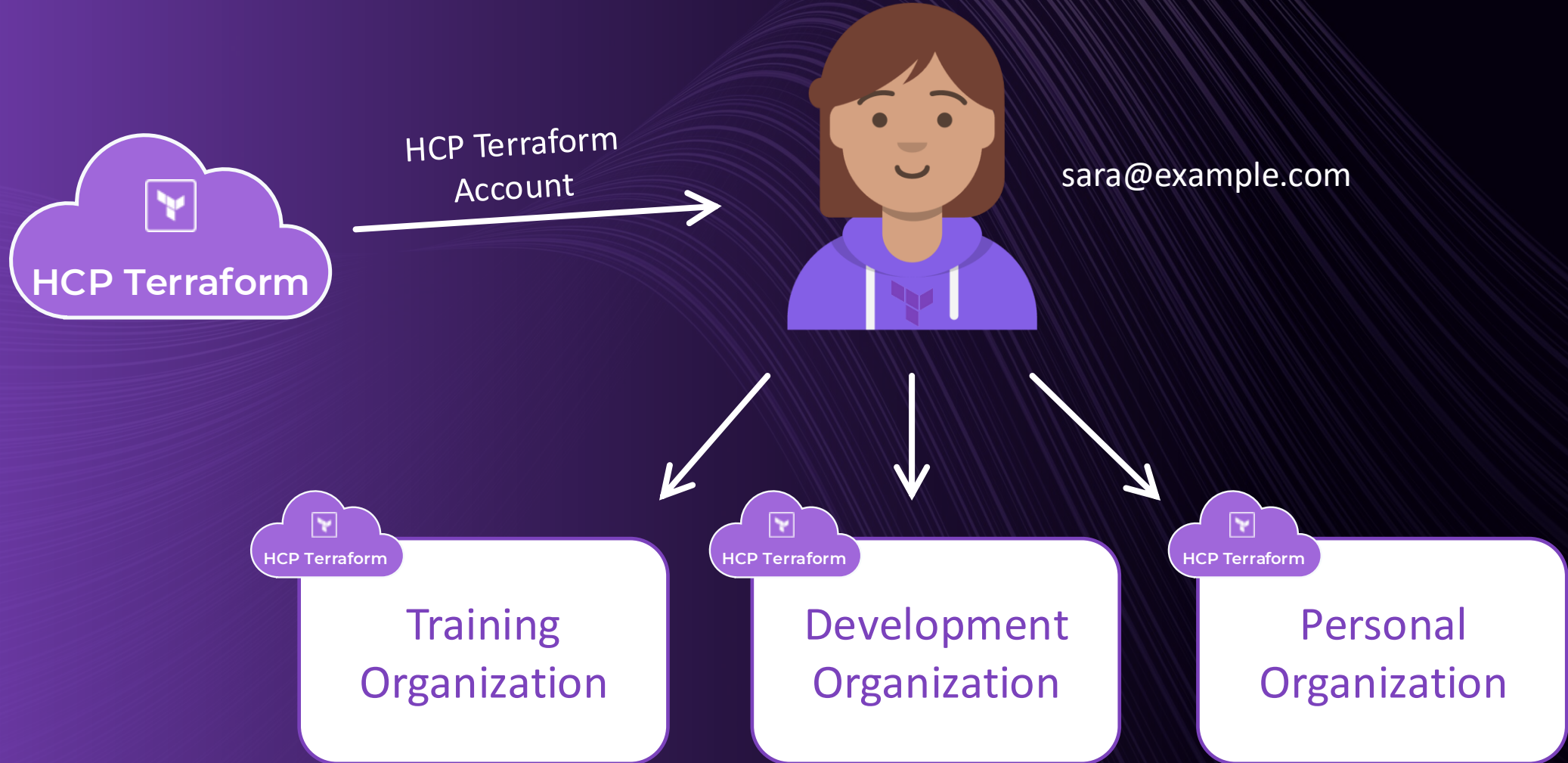
When two variable sets at the same scope conflict, HCP Terraform uses lexical (alphabetical) order of the set names

LECTURE

HCP Terraform Teams & Permissions



HCP Terraform Users



HCP Terraform Teams



The Access Control Hierarchy



Users

Individuals with HCP Terraform accounts

Can belong to multiple teams and multiple organizations



Teams

Groups of users within an organization

Granted permissions at the organization, project, and workspace levels



Permissions

Additive model - the highest permission always wins

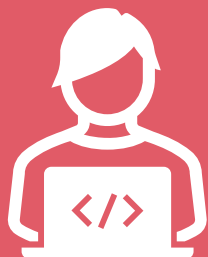
Follow the principle of least privilege

The Owners Team



Every Organization has a default Owners team that has full admin access to everything

Manage Policies



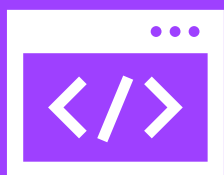
Manage Policy Overrides



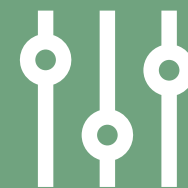
Manage Workspaces



Manage Private Registry



Manage VCS Settings



Manage Run Tasks



HCP Terraform Teams



Team Visibility

- Visible: Visible to every member of this organization
- Secret: Only visible to team members and organization owners

Team Tokens

- Team API tokens are designed for performing API operations on workspaces. They have the same access level to the workspaces the team has access to.

Organization, Project, & Workspace Permissions



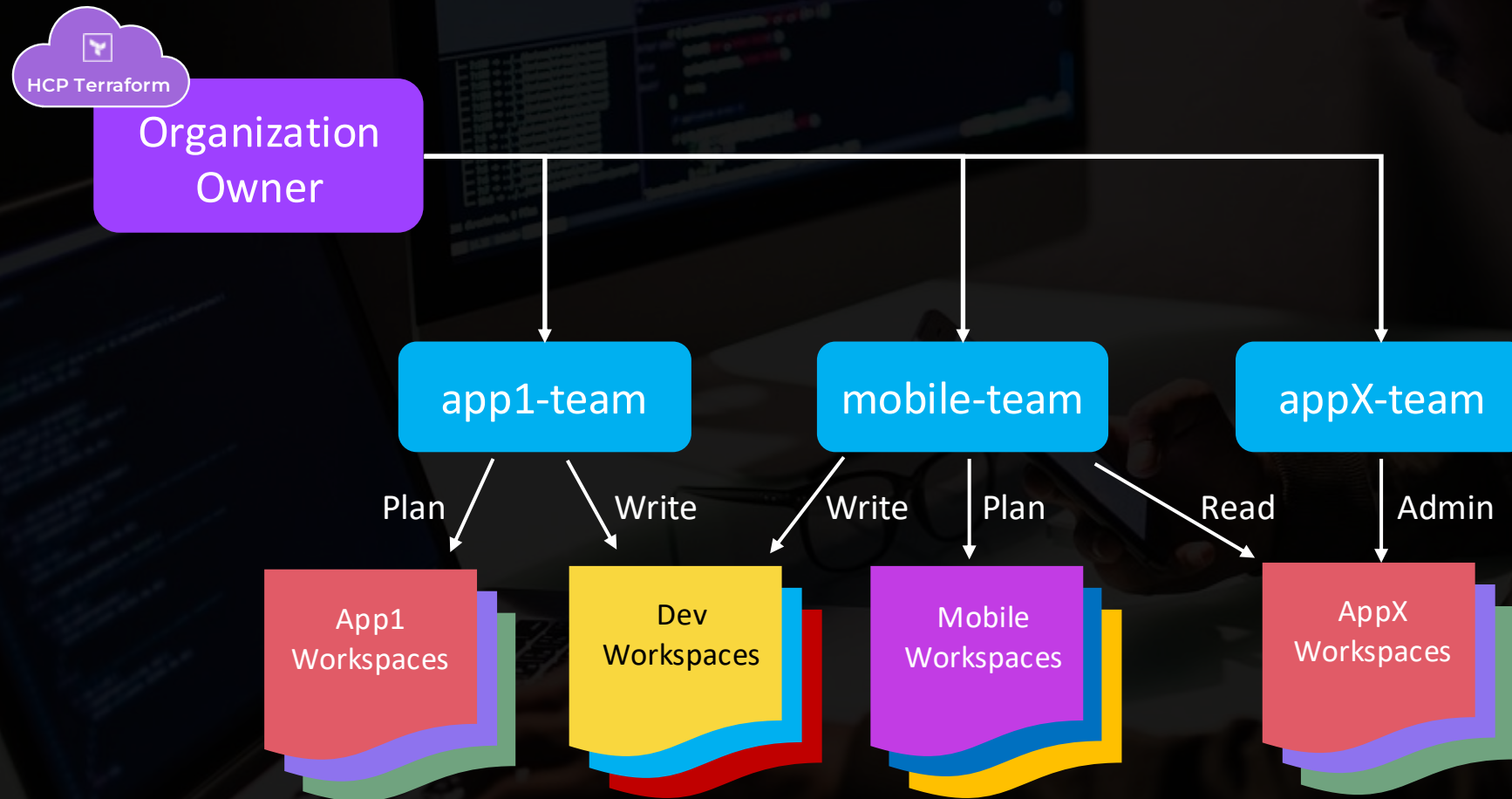
Broad permissions like Manage all workspaces, Manage policies, Manage VCS settings, View all workspaces

Permissions scoped to a project and all workspaces within it (Read, Write, Maintain, Admin, or Custom)

Permissions on a specific workspace such as Admin, Write, Plan, Read, or Custom

Permissions are additive
HCP Terraform always grants the highest permission level

Role Based Access Control (RBAC)

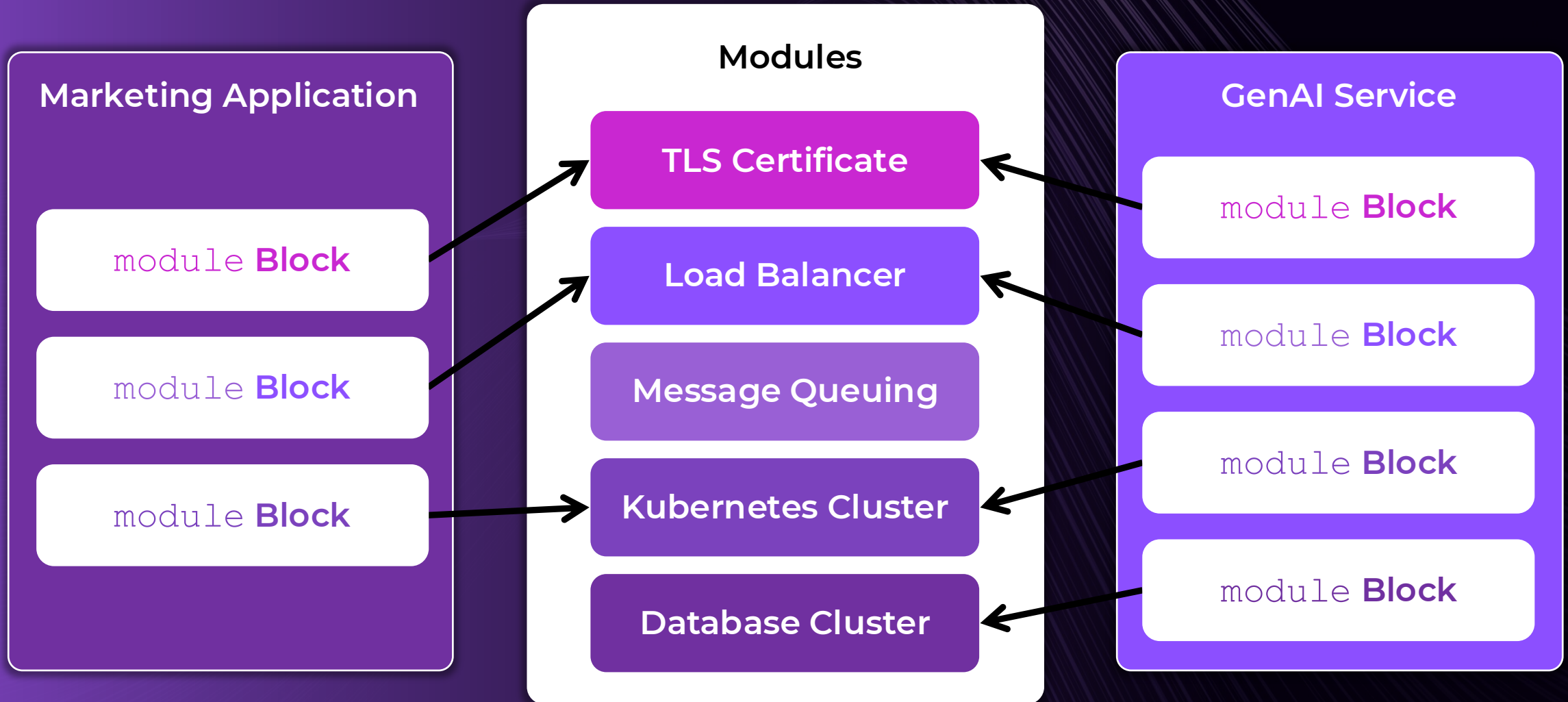


LECTURE

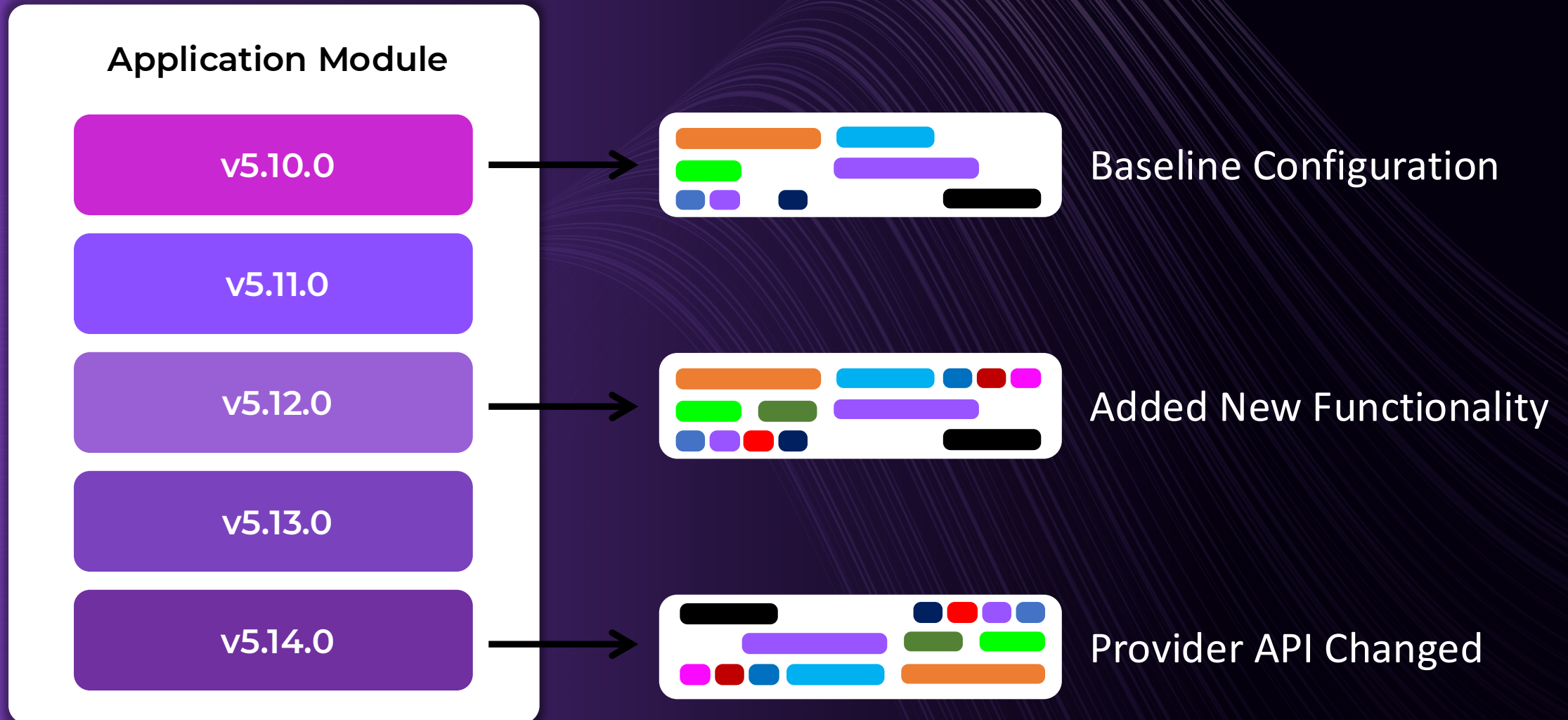
The HCP Terraform Private Registry



Modules for Repeatability



Module Versions





“ ”

**...what if we only want our
organization to see our
modules and not the
general public?**

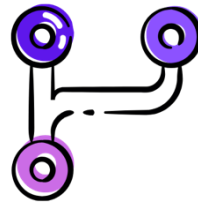
Private Registry



Share Modules Privately

Modules and providers are only available to members of your organization

They are not public



Version Controlled

Supports semantic versioning with version constraints, just like the public registry



Enforce Standardization

Develop approved modules and providers so teams build from tested, compliant components

Wait...This Looks Like the Public Registry



	Public Registry	Private Registry
Hosted At:	registry.terraform.io	Inside your HCP Terraform organization
Visibility	Anyone on the internet	Only members of your organization
Who Publishes Content	Anyone	Your team from your connected VCS repos
Content	Community modules & providers	Your private modules + curated public modules
Source Format	terraform-aws-modules/iam/aws	app.terraform.io/krausen-hcp/webapp/gcp

Same experience, different audience

What's Inside the Private Registry?



Private Modules & Providers

Published from your organization's VCS repos, only accessible to organization members



Curated Public Modules

Public registry modules/providers you've added to your private registry as "approved" for your org

Publishing Private Modules



01 VCS REPO



Module code lives in GitHub, GitLab, Bitbucket, etc. Must follow standard module structure.

02 HCP TERRAFORM



Publish to your private registry. New versions auto-created from Git tags (v1.0.0, v1.1.0).

03 ORG MEMBERS



Organization members browse, search, and reference modules in their configurations.

Using Private Modules (The Source Format)



Modules from the private registry can be referenced using a registry source address of the form:

`app.terraform.io/<ORGANIZATION-NAME>/terraform/<NAME>/<PROVIDER>`

```
1 module "webapp" {
2   source = "app.terraform.io/krausen-hcp/webapp/gcp"
3   name   = var.name
4   prefix = var.prefix
5   version = "1.0.0"
6 }
```



Key Takeaways

Private Registry

- ✧ The private registry lets you share modules and providers privately within your organization
- ✧ You can also curate approved public modules/providers alongside your private ones
- ✧ Private modules are published from VCS repositories and versioned with Git tags
- ✧ The source format includes the hostname:
`app.terraform.io/<ORG>/<MODULE>/<PROVIDER>`
- ✧ All org members can view and use the registry; publishing requires Manage Private Registry permissions

LECTURE

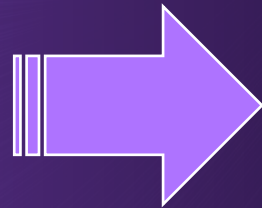
Policy Enforcement with Sentinel & OPA



Why Policy Enforcement?

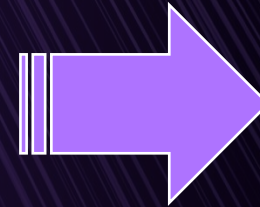


DEVELOP



terraform
apply

DEPLOY



Wrong
Policy

BREACH



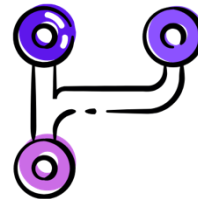
What if we could catch misconfigurations and policy violations before resources ever get deployed?

Policy as Code



Codified Rules

Security, compliance,
and operational
requirements written as
code



Version Controlled

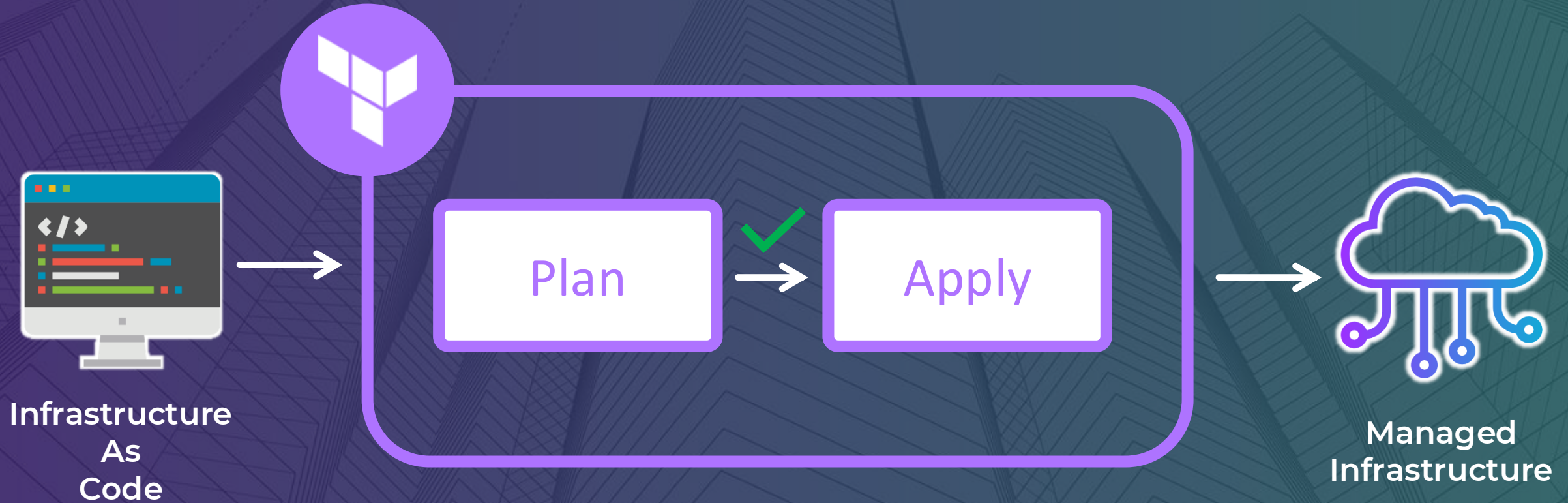
Stored in VCS, reviewed
through pull requests,
just like infrastructure
code



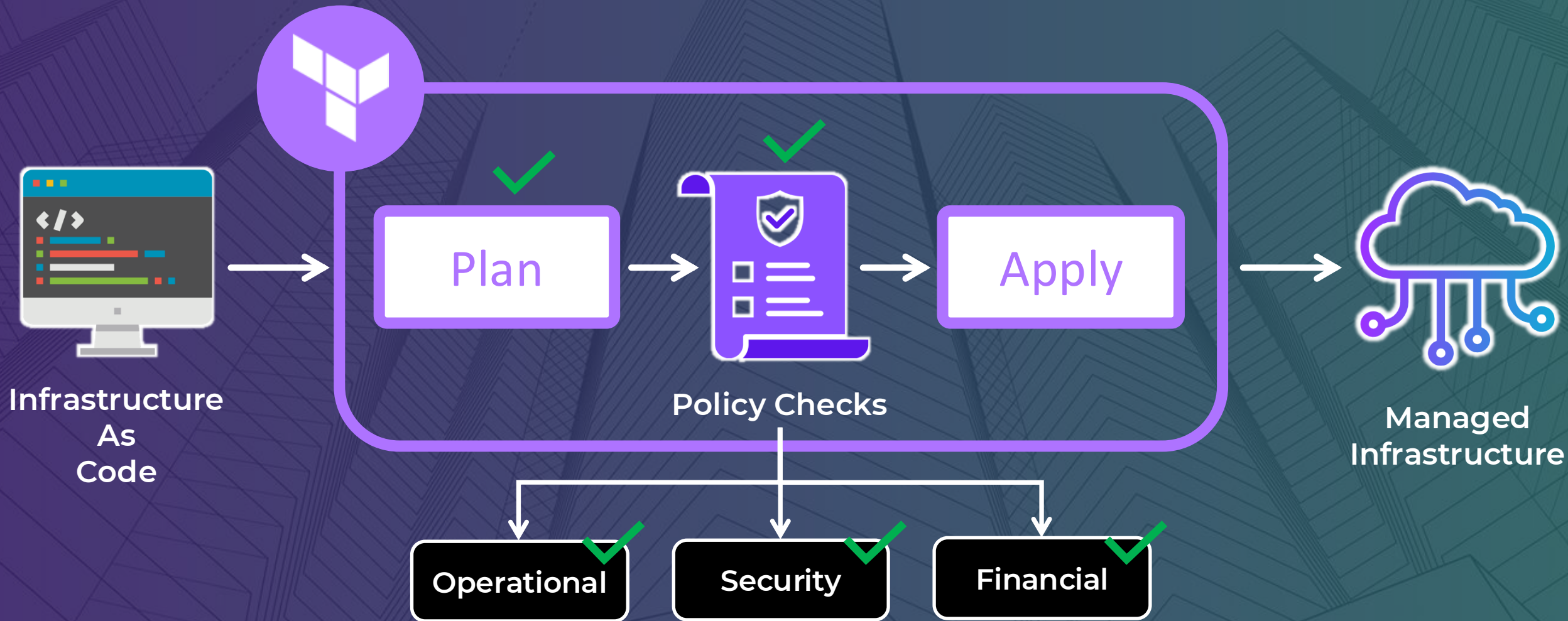
Automatically Enforced

Evaluated on every run,
no manual review
required

Terraform Run Before Policies



Terraform Run With Policies



Policy Check



Policies evaluate the plan data before any infrastructure is created



Failed policies can block the apply entirely



Sentinel policy checks also have access to cost estimation data



If the plan itself fails, policies are never evaluated

Two Frameworks: Same Goal



Sentinel

- ✧ Created and Maintained by HashiCorp
- ✧ Policy Language: Sentinel
- ✧ Ecosystem: HashiCorp Stack
- ✧ Pre-Written Policies: Yes, from HashiCorp

VS



Open Policy Agent (OPA)

- ✧ Created and Maintained by CNCF
- ✧ Policy Language: Rego
- ✧ Ecosystem: Cloud-Native
- ✧ Pre-Written Policies: Community-Driven

You can use both in the same workspace.



Sentinel

GCP Example

SSH is restricted from
the internet

```
1  import "tfplan/v2" as tfplan
2
3  allUnsupportedSourceRanges = [
4    "/0",
5    "0.0.0.0",
6    "0.0.0.0/0",
7  ]
8
9  allFirewallResources = filter tfplan.resource_changes as _, resource_changes {
10    resource_changes.type is "google_compute_firewall" and
11    resource_changes.mode is "managed" and
12    (resource_changes.change.actions contains "create" or
13     resource_changes.change.actions is ["update"])
14  }
15
16  print("Ensure that SSH access is restricted from the internet")
17
18  deny_public_ssh_access = rule {
19    all allFirewallResources as _, firewall {
20      all filter firewall.change.after.allow as _, allow {
21        allow.ports contains "22"
22      } as _ {
23        all firewall.change.after.source_ranges as _, range {
24          range not in allUnsupportedSourceRanges
25        }
26      }
27    }
28  }
29
30  main = rule {
31    deny_public_ssh_access
32  }
```



OPA

AWS Example

Checks whether
Security Groups
include Ingress from
0.0.0.0/0

```
1 package terraform.policies.public_ingress
2
3 import input.plan as plan
4
5 deny[msg] {
6     r := plan.resource_changes[_]
7     r.type == "aws_security_group"
8     r.change.after.ingress[_].cidr_blocks[_] == "0.0.0.0/0"
9     msg := sprintf("%v has 0.0.0.0/0 as allowed ingress", [r.address])
10 }
```


Sentinel Enforcement Levels



Advisory

- ✦ Failed policies produce a warning but the run continues
 - ✦ Use for new policies you're introducing but not yet enforcing
-



Soft Mandatory

- ✦ Failed policies stop the run, but can be overridden
 - ✦ Requires the Manage Policy Overrides permission to override
 - ✦ All overrides are logged for auditing
-



Hard Mandatory

- ✦ Failed policies stop the run and cannot be overridden
- ✦ The violation must be fixed in the configuration before the run can proceed
- ✦ The default enforcement level for Sentinel policies

OPA Enforcement Levels



Advisory

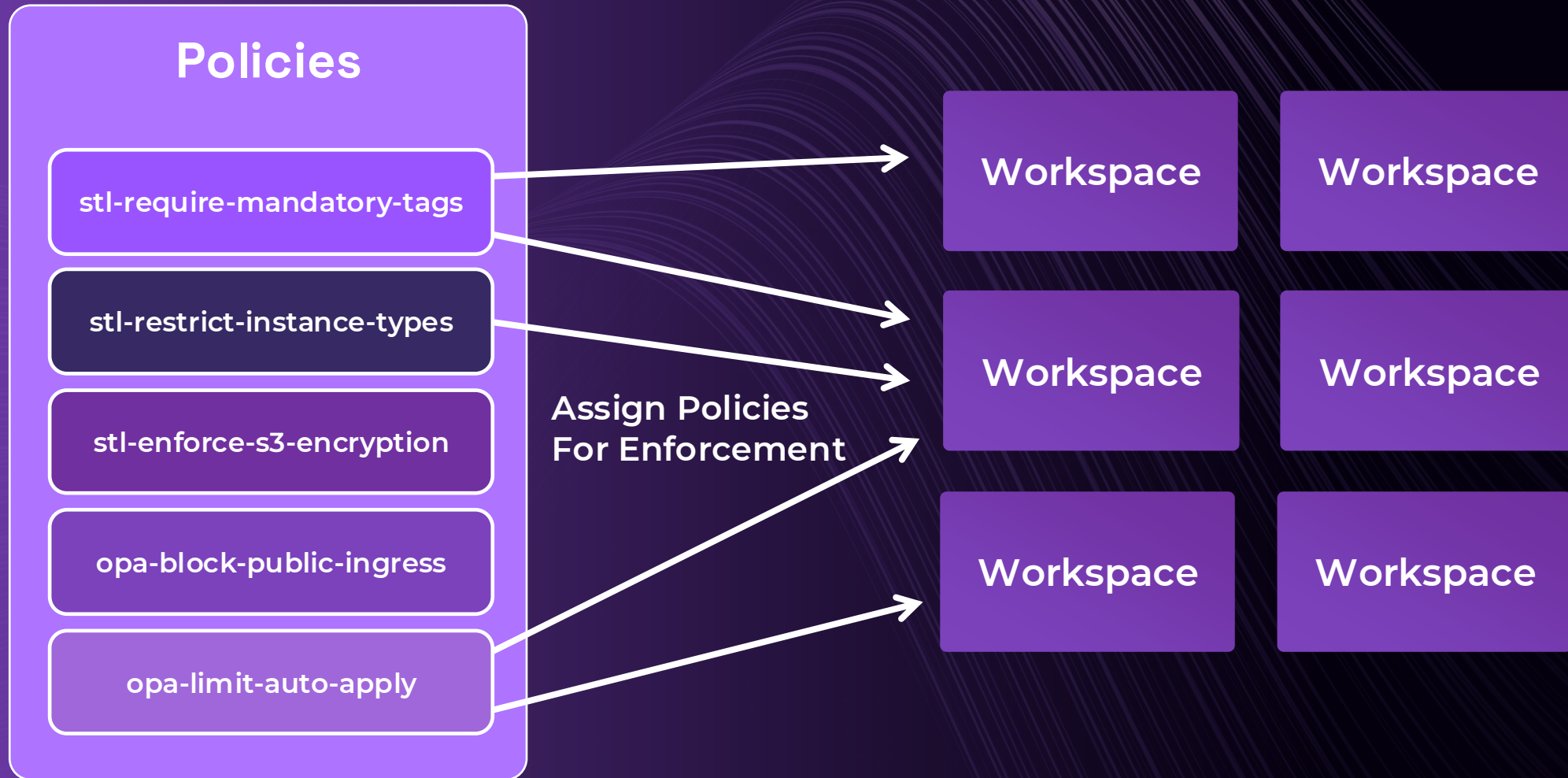
- ✦ Failed policies produce a warning but the run continues
- ✦ Use for new policies you're introducing but not yet enforcing



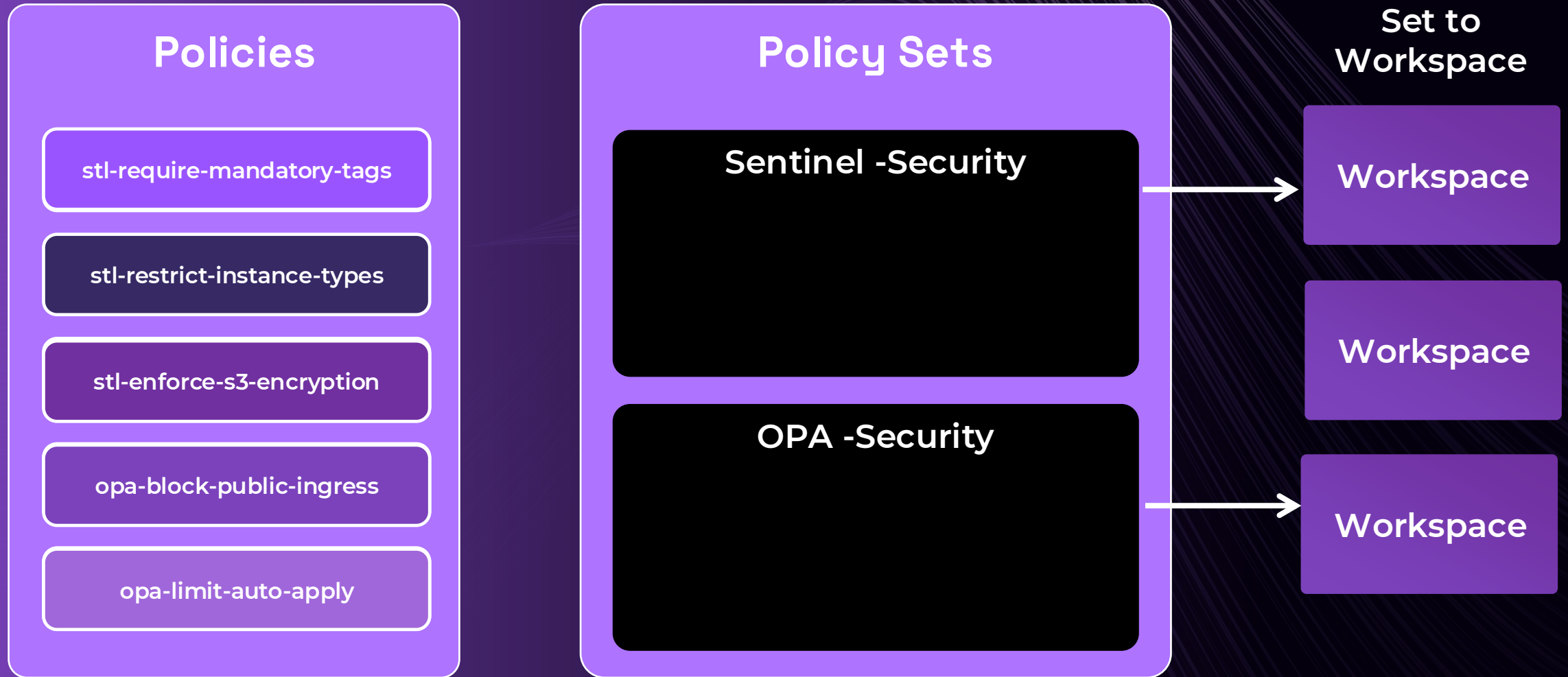
Mandatory

- ✦ Failed policies stop the run and can be overridden
- ✦ The violation must be fixed in the configuration before the run can proceed

Develop Policies



Policy Sets



Policy Sets



Policies Are Grouped
Into Policy Sets



Each Set Is One
Framework Only
(All Sentinel or All OPA)



Sets Can Be Scoped
Globally, to Projects, or
Specific Workspaces



You Can Exclude
Workspaces From A Set



Three Management
Methods (UI, VCS, API)

What Policies Can and Can't Do



Examples of What Policies Can Do

- ✓ Block Public S3 Buckets
- ✓ Restrict Instance Types
- ✓ Enforce Tagging
- ✓ Limit Regions
- ✓ Enforce Cost Measures

What Policies Can NOT Do

- ✗ Continuously Monitor After Deployment
- ✗ Remove Existing Violating Resources
- ✗ Validate That Attribute Values Are Valid
- ✗ Analyze Runtime Behavior



Key Takeaways

Policy Enforcement

- ✦ Policies evaluate after the plan phase but before apply, catching issues before infrastructure is created
- ✦ HCP Terraform supports two policy frameworks: Sentinel (HashiCorp) and OPA (open source, uses Rego)
- ✦ Sentinel has three enforcement levels (advisory, soft mandatory, hard mandatory) while OPA has two (advisory, mandatory)
- ✦ Policy sets can be scoped globally, to specific projects, or to individual workspaces
- ✦ Overriding a failed policy requires the Manage Policy Overrides permission

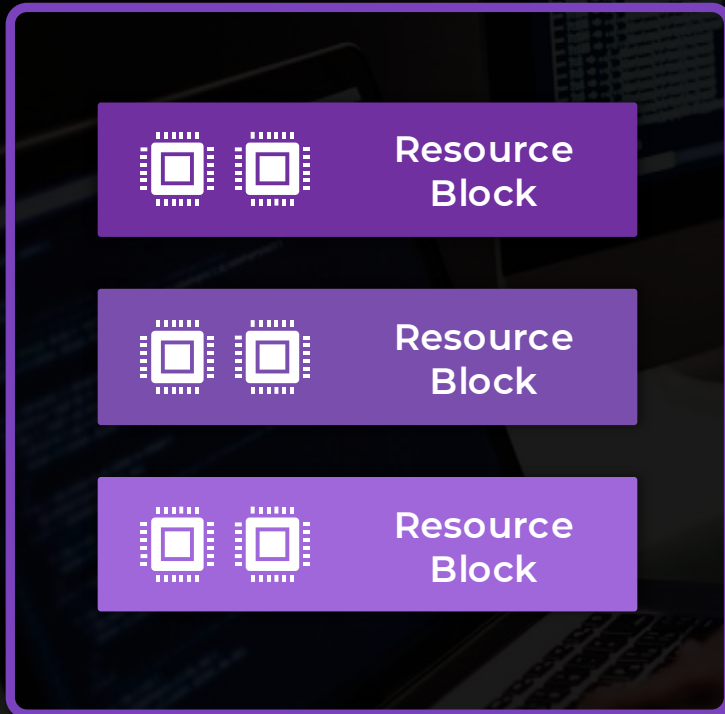
LECTURE

Additional HCP Terraform Workspace Features

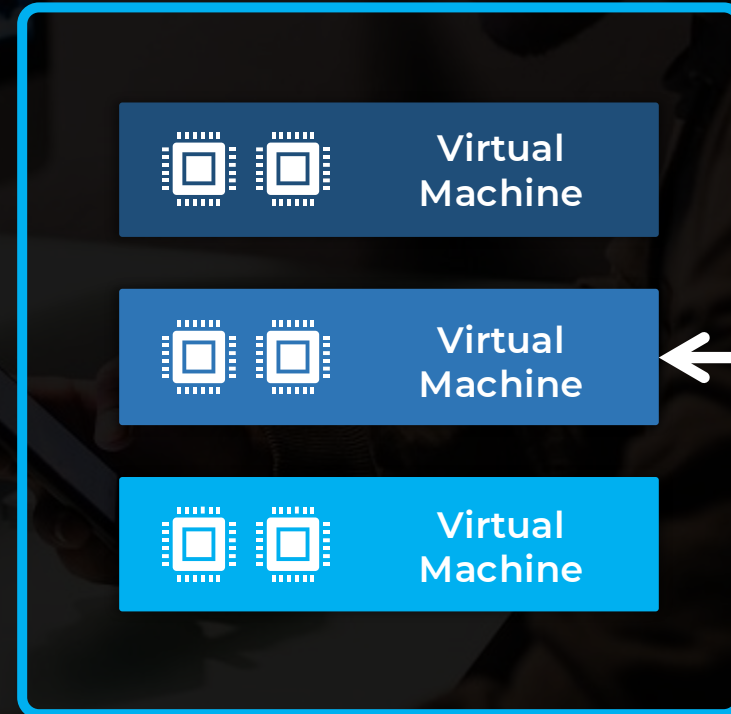


Health Assessments

Terraform Configuration
(Desired State)



Azure Subscription
(Production)



Makes Change
in Azure Portal

How Do You Know Your Infrastructure Is Still Correct?

Health Assessments



Once enabled, HCP Terraform can automatically run health assessments on your workspaces once every 24 hours or so

Drift Detection

Compares Real Infrastructure Against Your State File

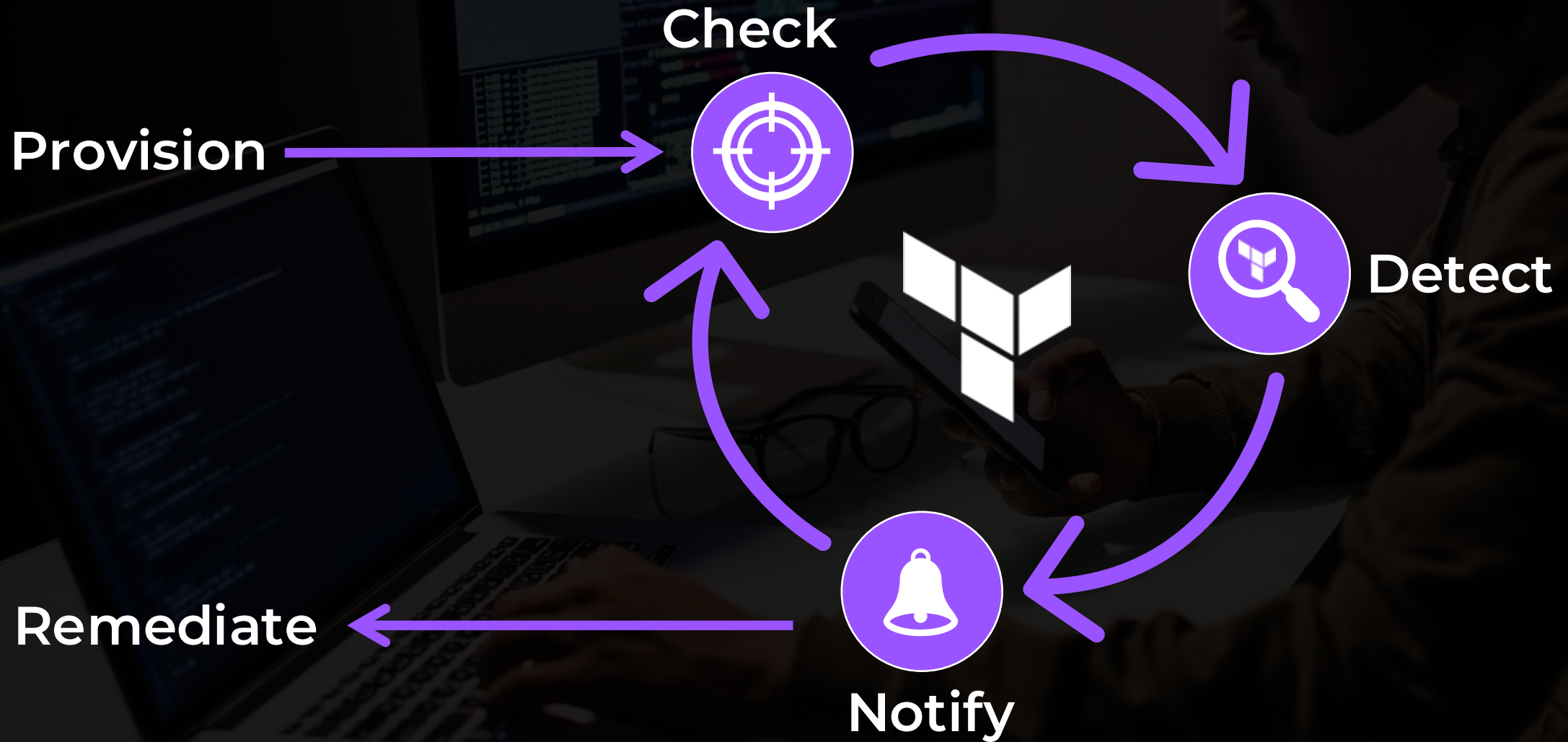
Did Something Change Outside Terraform?

Continuous Validation

Evaluates `check` Blocks, Preconditions, And Postconditions

Are Your Custom Rules Still Passing?

Health Assessments

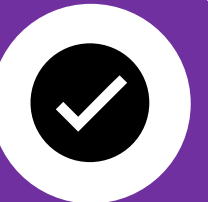
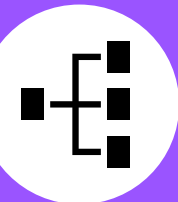


Health Assessments



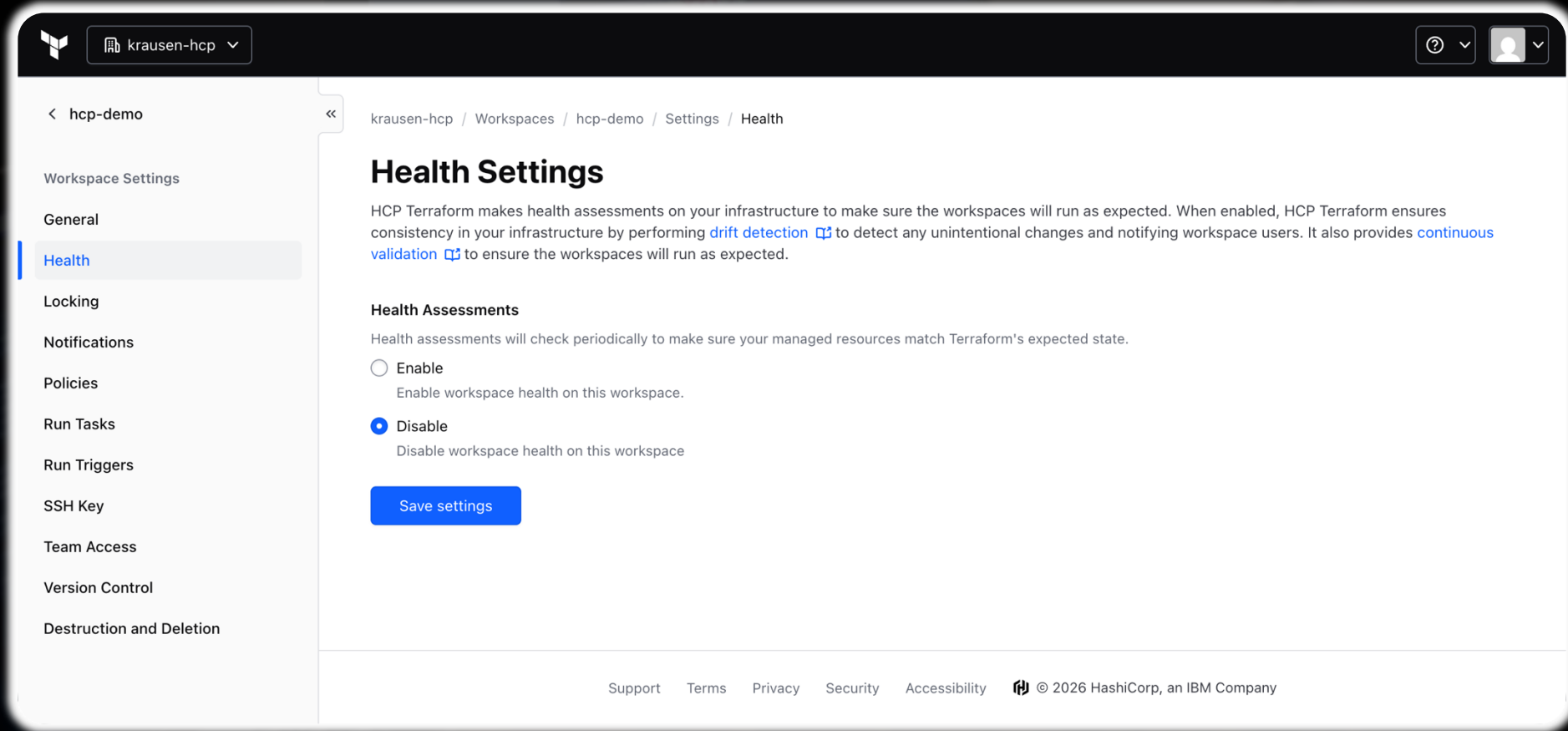
If something's wrong, your workspace shows a "Health warning" status

Health assessments are available in HCP Terraform Standard and Premium editions



Can be enabled per-workspace or enforced across the entire organization

Health Assessments

A screenshot of the HashiCorp HCP Terraform web interface showing the 'Health Settings' page for a workspace named 'hcp-demo'. The page has a dark header with the HashiCorp logo, a workspace selector 'krausen-hcp', and user profile icons. A left sidebar lists various settings categories, with 'Health' selected. The main content area is titled 'Health Settings' and contains a paragraph explaining that HCP Terraform performs drift detection and continuous validation. Below this, the 'Health Assessments' section shows two radio button options: 'Enable' and 'Disable'. The 'Disable' option is currently selected. A blue 'Save settings' button is at the bottom of this section. The footer of the page contains links for Support, Terms, Privacy, Security, and Accessibility, followed by a copyright notice for HashiCorp, an IBM Company.

 krausen-hcp

< hcp-demo

Workspace Settings

General

Health

Locking

Notifications

Policies

Run Tasks

Run Triggers

SSH Key

Team Access

Version Control

Destruction and Deletion

krausen-hcp / Workspaces / hcp-demo / Settings / Health

Health Settings

HCP Terraform makes health assessments on your infrastructure to make sure the workspaces will run as expected. When enabled, HCP Terraform ensures consistency in your infrastructure by performing [drift detection](#) to detect any unintentional changes and notifying workspace users. It also provides [continuous validation](#) to ensure the workspaces will run as expected.

Health Assessments

Health assessments will check periodically to make sure your managed resources match Terraform's expected state.

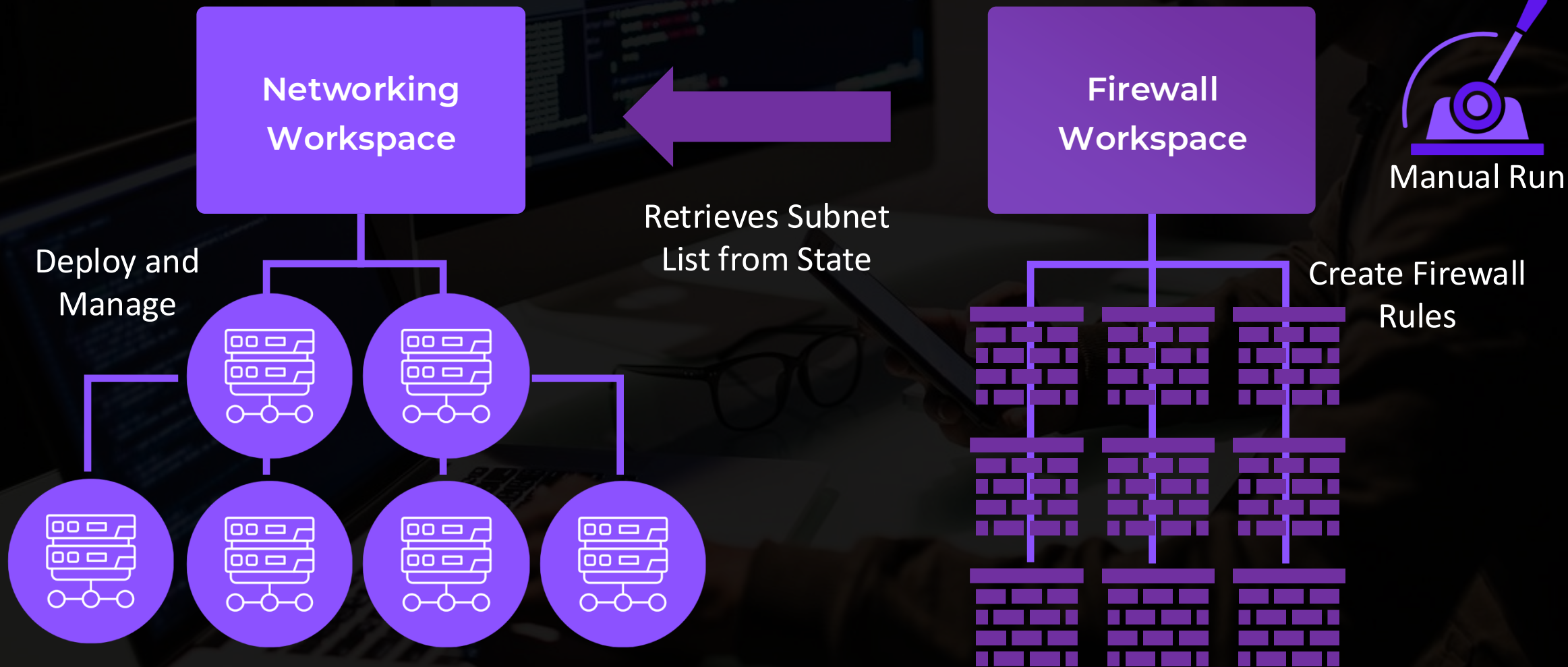
☐ Enable
Enable workspace health on this workspace.

☒ Disable
Disable workspace health on this workspace

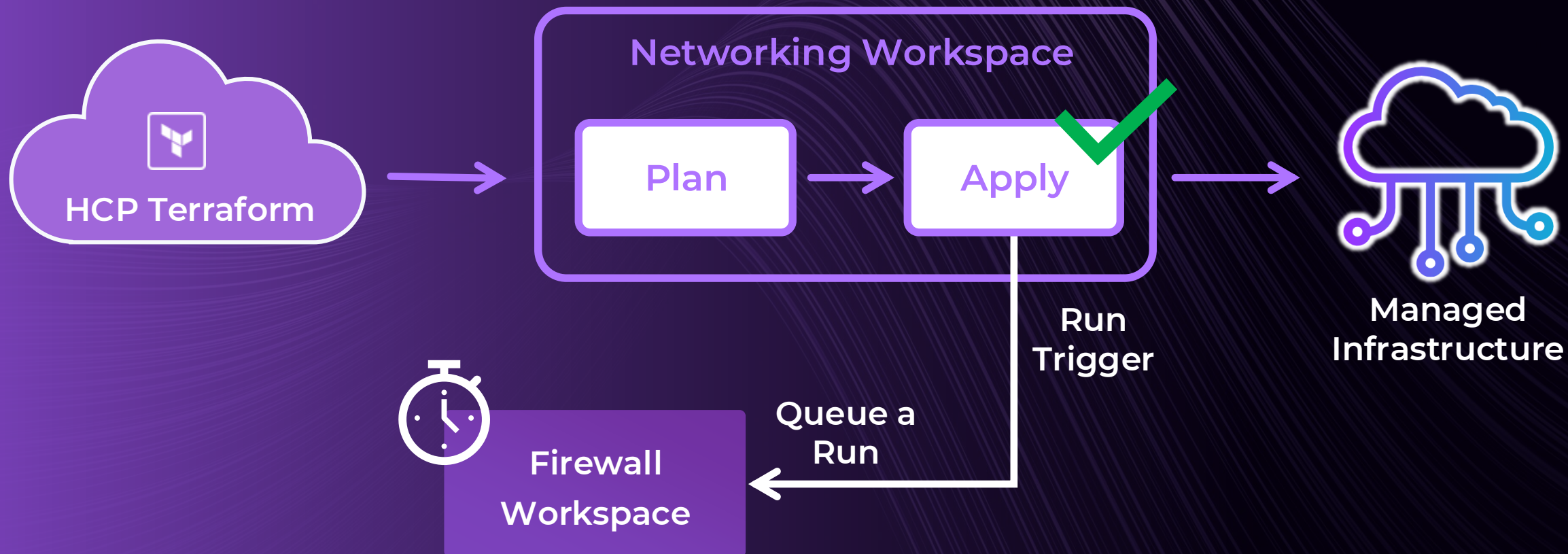
Save settings

Support Terms Privacy Security Accessibility © 2026 HashiCorp, an IBM Company

Workspaces Often Depend On Each Other



Automating Runs with Run Triggers



Benefits of Run Triggers



Run Triggers let you connect a workspace to one or more source workspaces



HCP Terraform automatically queues a run in the connected downstream workspace



Each workspace can connect to up to 20 source workspaces



Run triggers fire on successful applies in source workspaces and that they enable workspace-to-workspace automation



Creating a Run Triggers



 krausen-hcp ▾

 ▾  ▾

< hcp-demo

Workspace Settings

General

Health

Locking

Notifications

Policies

Run Tasks

Run Triggers

SSH Key

Team Access

Version Control

Destruction and Deletion

<<

krausen-hcp / Workspaces / hcp-demo / Settings / Run Triggers

>>

Run Triggers

Run triggers allow you to connect this workspace to one or more source workspaces. These connections allow runs to queue automatically in this workspace on successful apply of runs in any of the source workspaces.

Auto-apply

Sets this workspace to automatically apply changes for successful runs. If disabled, runs require operator approval.

☐ Auto-apply run triggers

Automatically apply changes when a source workspace successfully applies.

Connected workspaces [Connect workspace](#)

Name	Privileges
networking-prod	Connected × Disconnect

1-1 of 1

< 1 >

Items per page 50 ▾

Tracking Work



- ❧ As your organization scales, you'll have hundreds of workspaces
- ❧ Some might need updates, version, upgrades, or fixes
- ❧ As an HCP Terraform admin, you need the ability to add to-do items you can attach to workspaces



Tracking Work with Change Requests



Change Requests are actionable "to-do" items you can attach to workspaces



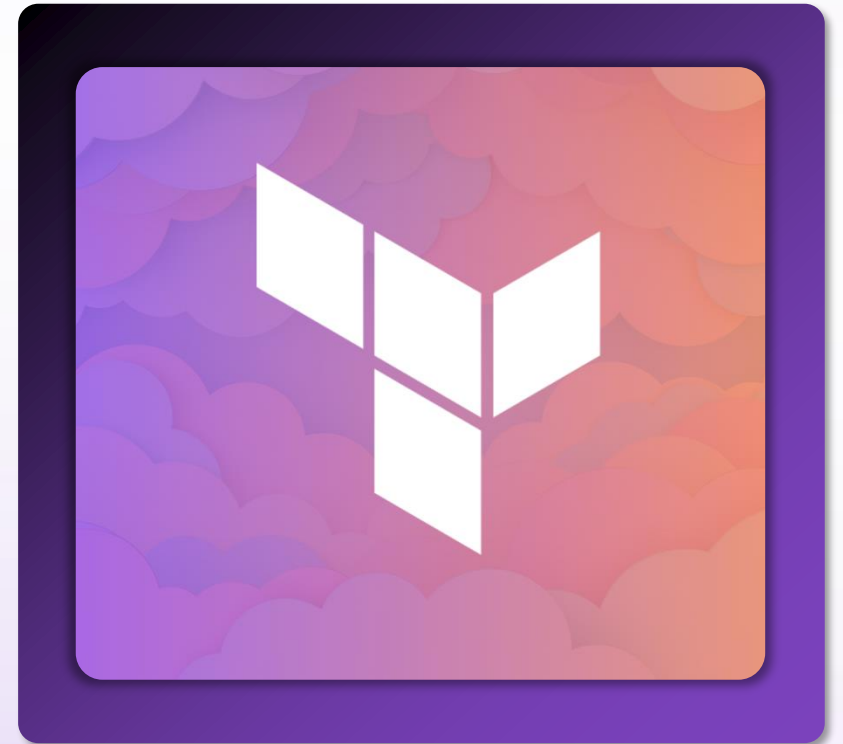
You add the messages directly from the Explorer and they get tracked for each workspace



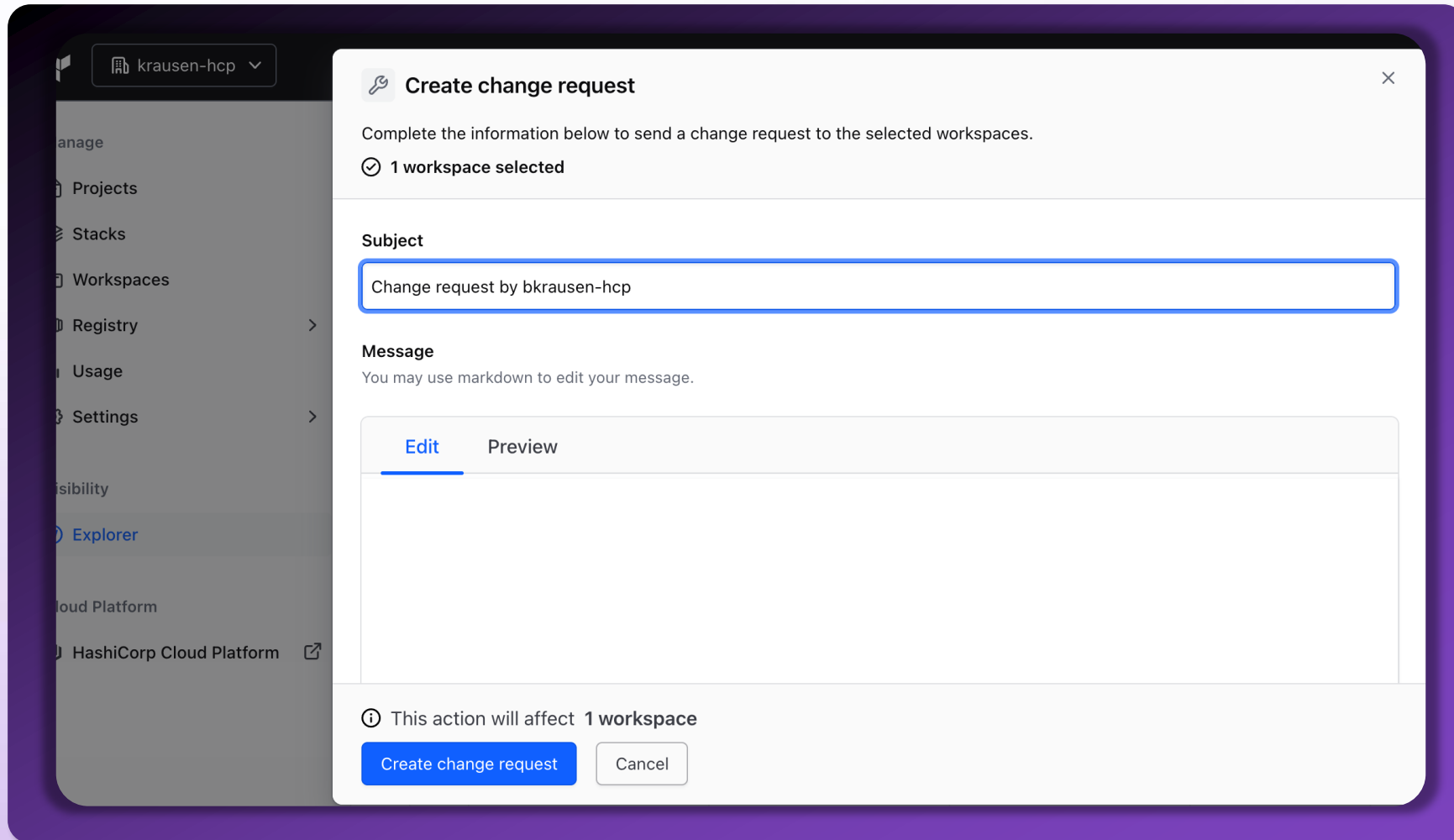
Teams can be notified when new change requests are created on their workspaces



Available in Standard and Premium editions



Tracking Work with Change Requests



The screenshot shows the HashiCorp Terraform Cloud interface. On the left is a sidebar with navigation links: Manage, Projects, Stacks, Workspaces, Registry, Usage, Settings, Visibility, Explorer, Cloud Platform, and HashiCorp Cloud Platform. The main area displays a 'Create change request' dialog. The dialog has a title bar with a wrench icon and a close button. Below the title bar, it says 'Complete the information below to send a change request to the selected workspaces.' and '1 workspace selected'. The 'Subject' field is highlighted with a blue border and contains the text 'Change request by bkrausen-hcp'. The 'Message' field is below it, with a note 'You may use markdown to edit your message.' and tabs for 'Edit' and 'Preview'. At the bottom, there is a warning icon and text 'This action will affect 1 workspace', followed by 'Create change request' and 'Cancel' buttons.

Create change request

Complete the information below to send a change request to the selected workspaces.

✓ 1 workspace selected

Subject

Change request by bkrausen-hcp

Message

You may use markdown to edit your message.

[Edit](#) [Preview](#)

ⓘ This action will affect 1 workspace

[Create change request](#) [Cancel](#)

Gaining Visibility with the Explorer



The screenshot displays the HashiCorp Explorer interface for the organization 'krausen-hcp'. The left sidebar contains a navigation menu with sections: Manage (Projects, Stacks, Workspaces, Registry, Usage, Settings), Visibility (Explorer, highlighted with a purple arrow), and Cloud Platform (HashiCorp Cloud Platform). The main content area is titled 'Explorer' and shows the 'Types & use cases' view. It includes a 'New query' button and a list of types: Modules, Providers, Workspaces, and Terraform Versions. Below this, the 'Use cases' section lists pre-defined views: Top module versions, Latest Terraform versions, Top provider versions, Workspaces without VCS, Workspace VCS source, and Workspaces with failed checks. Each item has a right-pointing arrow icon.